

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

ENGENHARIA DE COMPUTAÇÃO

DAVI CELESTINO DOS SANTOS BARBOSA, 23210541

JOÃO VICTOR TENÓRIO VENANCIO, 23211191

OTAVIO JOSHUA COSTA BRANDAO MENEZES, 23211189

LARISSA FERREIRA DIAS DE SOUZA, 23211102

**RELATÓRIO TÉCNICO: SISTEMA DE RECOMENDAÇÃO BASEADO EM
ONTOLOGIAS**

MACEIÓ - AL

2026

UNIVERSIDADE FEDERAL DE ALAGOAS

CAMPUS A.C. SIMÕES

ENGENHARIA DE COMPUTAÇÃO

DAVI CELESTINO DOS SANTOS BARBOSA, 23210541

JOÃO VICTOR TENÓRIO VENANCIO, 23211191

OTAVIO JOSHUA COSTA BRANDAO MENEZES, 23211189

LARISSA FERREIRA DIAS DE SOUZA, 23211102

**RELATÓRIO TÉCNICO: SISTEMA DE RECOMENDAÇÃO BASEADO EM
ONTOLOGIAS**

Este relatório descreve a concepção, modelagem e avaliação de um Sistema de Recomendação semântico, desenvolvido com base nos princípios da Web Semântica, utilizando a linguagem OWL 2.0 e processamento lógico via motor de inferência (Reasoner).

Orientador (a): Prof. Dr. Evandro de Barros Costa

MACEIÓ - AL

2026

SUMÁRIO

1. DESCRIÇÃO DO DOMÍNIO.....	4
2. MODELAGEM CONCEITUAL DA ONTOLOGIA.....	4
3. JUSTIFICATIVA DAS PRINCIPAIS CLASSES E PROPRIEDADES.....	4
4. EXEMPLOS DE CONSULTAS E INFERÊNCIAS.....	5
5. DESCRIÇÃO DO MECANISMO DE RECOMENDAÇÃO.....	5
6. ANÁLISE CRÍTICA DAS VANTAGENS E LIMITAÇÕES DA ABORDAGEM.....	6
Vantagens.....	6
Limitações.....	6

1. DESCRIÇÃO DO DOMÍNIO

O domínio escolhido para a aplicação foi o de Recomendação Cinematográfica (Filmes). Este domínio é altamente propício para a modelagem ontológica devido à rica rede de relacionamentos entre suas entidades (filmes, pessoas, gêneros) e o perfil de consumo dos usuários.

O objetivo do sistema é atuar como um agente de recomendação que não depende de algoritmos estatísticos tradicionais (como filtragem colaborativa), mas sim de raciocínio lógico e dedutivo. Ao cruzar os gostos explícitos de um usuário com os metadados dos filmes, o sistema é capaz de deduzir matematicamente quais obras são adequadas para aquele perfil.

2. MODELAGEM CONCEITUAL DA ONTOLOGIA

A ontologia foi modelada utilizando uma hierarquia de classes (Taxonomia) fundamentada na classe raiz Thing (padrão OWL). A arquitetura conceitual foi dividida nos seguintes eixos:

- Eixo de Conteúdo: Representado pela classe Filme e suas subclasses especializadas (FilmeAcao, FilmeFiccao, FilmeDrama, etc.).
- Eixo de Entidades Nomeadas (Pessoas): Representado pela superclasse Pessoa, que se bifurca em Ator e Diretor, permitindo que uma mesma pessoa física possa pertencer a ambas as subclasses dependendo de seus relacionamentos.
- Eixo de Classificação: Representado pela classe Genero, utilizada para categorizar os filmes e os gostos dos usuários.
- Eixo de Consumo: Representado pela classe Usuario, que atua como o nó central para o qual as inferências de recomendação convergem.

3. JUSTIFICATIVA DAS PRINCIPAIS CLASSES E PROPRIEDADES

Para viabilizar a descoberta de conhecimento implícito, foram estabelecidas Object Properties (que ligam instâncias a instâncias) e Data Properties (que ligam instâncias a valores primitivos).

Principais Object Properties:

- temGenero e gostaDeGenero: Fundamentais para cruzar o perfil do Usuario com as características do Filme.
- temAtor e gostaDeAtor: Permitem recomendações baseadas em afinidade com o elenco.
- atuouEm e dirigiu: Definidas explicitamente como propriedades inversas de temAtor e temDiretor. Isso significa que, se declararmos que um filme tem um ator X, a ontologia automaticamente infere que o ator X atuou naquele filme, garantindo integridade bidirecional.

- recomendadoPara: A propriedade alvo do sistema. Seu domínio é o Usuario e seu contradomínio (range) é o Filme. Ela começa vazia e é preenchida dinamicamente pelo motor de inferência.

Principais Data Properties:

- Atributos como notaIMDB, anoLancamento e idadeUsuario foram incluídos para enriquecer a base de conhecimento, abrindo margem para regras futuras mais complexas (ex: recomendar apenas filmes com nota superior a 8.0 para determinados usuários).

4. EXEMPLOS DE CONSULTAS E INFERÊNCIAS

O uso do motor de raciocínio (Hermit) demonstrou forte capacidade de inferência sobre a base estruturada, destacando-se em duas operações:

1. Classificação Automática (Axiomas de Equivalência):
2. No código, a instância do filme Matrix foi declarada genericamente apenas como pertencente à classe Filme. Porém, foi definido o axioma lógico: `FilmeAcao.equivalent_to = [Filme & temGenero.some(onto.Genero("Acao"))]`. Ao rodar o reasoner, o sistema analisou que Matrix possui a relação temGenero com "Ação" e, automaticamente, reclassificou a instância para a subclasse FilmeAcao.
3. Propriedades Inversas:
4. Ao declarar que A Origem (Inception) temDiretor Christopher Nolan, o motor automaticamente vinculou que Christopher Nolan dirigiu A Origem, sem que nenhuma linha de código em Python precisasse iterar sobre isso.

5. DESCRIÇÃO DO MECANISMO DE RECOMENDAÇÃO

O mecanismo de recomendação é o coração do sistema e foi implementado de forma puramente declarativa através de Regras SWRL (Semantic Web Rule Language), e não através de condicionais procedurais (if/else).

As regras processadas pelo reasoner foram:

- Regra 1 (Afinidade por Gênero): `Usuario(?u), Filme(?f), gostaDeGenero(?u, ?g), temGenero(?f, ?g) -> recomendadoPara(?u, ?f)`
- Regra 2 (Afinidade por Elenco): `Usuario(?u), Filme(?f), gostaDeAtor(?u, ?a), temAtor(?f, ?a) -> recomendadoPara(?u, ?f)`

Funcionamento: Durante a etapa de realize (execução do Hermit), o motor busca padrões no grafo que satisfaçam as premissas (lado esquerdo da regra). Quando encontra um match (ex: O usuário João gosta de Keanu Reeves, e o filme John Wick tem Keanu Reeves no elenco), ele dispara a conclusão (lado direito) e injeta a relação recomendadoPara(João, John Wick) permanentemente na ontologia. O script em Python atua apenas como uma interface de exibição, iterando sobre a propriedade recomendadoPara após o reasoner finalizar seu trabalho.

6. ANÁLISE CRÍTICA DAS VANTAGENS E LIMITAÇÕES DA ABORDAGEM

Vantagens

1. Explainable AI (Inteligência Artificial Explicável): Diferente de redes neurais (caixas-pretas), o sistema baseado em ontologias permite extrair a justificativa exata de uma recomendação. É possível cruzar o porquê da regra ter sido ativada, gerando transparência para o usuário final.
2. Separação entre Lógica e Código: As regras de negócio e relacionamentos vivem na ontologia (.owl). O código Python atua apenas como orquestrador. Se a lógica de recomendação mudar, altera-se o arquivo semântico, não o software.
3. Descoberta de Conhecimento: A capacidade de classificar indivíduos dinamicamente reduz a necessidade de categorização manual exaustiva do banco de dados.

Limitações

1. Escalabilidade Computacional: A execução de reasoners (como Pellet ou HermiT) em regras complexas (SWRL) sobre bases de dados massivas sofre de complexidade exponencial. Para sistemas em tempo real do tamanho da Netflix, a inferência semântica pura seria lenta demais sem otimizações ou fragmentação de grafos.
2. Rigidez Lógica (Falta de Pesos): A lógica descritiva é estrita (Verdadeiro ou Falso). O sistema atual tem dificuldade em representar nuances como "O usuário gosta muito de Ação, mas gosta pouco de Drama". Incorporar lógica Fuzzy ou pesos probabilísticos em OWL 2.0 requer extensões não-padronizadas e complexas, área onde algoritmos de Machine Learning estatístico possuem vasta vantagem.
3. Open World Assumption (OWA): A suposição de mundo aberto do OWL exige cuidado. Se não dissermos que um filme NÃO é de comédia, o reasoner assume que ele pode ser, o que pode gerar comportamentos inesperados caso as classes não possuam restrições disjointWith (classes disjuntas) bem definidas.

Se precisar de qualquer outro ajuste, formatação diferente ou alguma ajuda para fechar o envio final do seu projeto, estou à disposição! Foi um excelente trabalho.